SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA0606 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:100**

Annexure A

Experiments

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

3. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

4. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

5. Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS TO EXPERIMENTS (ANNEXURE A)

Question 1

Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

Component	Answer Details
Aim	To design and conduct an experiment that implements a Naïve Bayes classifier on a real-world dataset, and to study the effect of feature sets, the feature-independence assumption, and Laplace smoothing on classification performance.
Dataset Used	UCI Adult Income / SMS Spam Collection dataset (categorical and text features such as age-group, occupation, word-frequency tokens). 80% of records are used for training and 20% for testing, using stratified sampling so that class proportions are preserved.
Experimental Design	Three feature sets are constructed: (i) a minimal set of 3 attributes, (ii) a moderate set of 6 attributes, and (iii) the full attribute set. For each feature set the Naïve Bayes model is trained twice – once using raw Maximum Likelihood probability estimates and once using Laplace (add-one) smoothed estimates – giving six trained models in total.
Procedure	1. Load and pre-process the dataset (handle missing values, discretize continuous attributes). 2. Compute class-conditional probabilities $P(x_i\|C)$ for every attribute value using the Naïve Bayes independence assumption $P(C\|X) \propto P(C)\ \Pi P(x_i\|C)$. 3. Train the model separately on each of the three feature sets. 4. For each feature set, repeat training with Laplace smoothing $P(x_i\|C) = (\text{count} + 1) / (N_C + k)$. 5. Predict class labels on the held-out test set and record the confusion matrix for every model. 6. Compute Accuracy, Precision and Recall from each confusion matrix.
Sample Results	Minimal feature set: 78% accuracy (no smoothing) vs 81% (with smoothing). Moderate feature set: 85% accuracy (no smoothing) vs 88% (with smoothing). Full feature set: 89% accuracy (no smoothing) vs 92% (with smoothing), with precision improving from 0.83 to 0.90 and recall from 0.80 to 0.89.
Analysis – Effect of Feature Independence	Accuracy rises as more features are added, confirming that additional evidence improves the posterior estimate even though attributes such as occupation and education are not truly independent. The conditional-independence assumption causes a small, consistent bias because correlated attributes are double-counted, but it does not prevent the classifier from ranking classes correctly, which is why Naïve Bayes remains competitive despite the simplifying assumption.

Analysis – Effect of Laplace Smoothing	Without smoothing, any attribute value that never appears with a class in the training data receives a zero probability, which forces the entire posterior product to zero regardless of other evidence. Laplace smoothing removes these zero probabilities by adding a pseudo-count of 1 to every event, which is why accuracy, precision and recall consistently improve after smoothing is applied, most noticeably for the smaller feature sets where zero-frequency values are more common.
Conclusion	The experiment confirms that Naïve Bayes performance improves with richer feature sets and that Laplace smoothing is essential for stable probability estimation, since it prevents zero-frequency problems and yields consistently higher accuracy, precision and recall across all feature-set configurations.

Question 2

Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

Component	Answer Details
Aim	To estimate the mean and variance of a Gaussian-distributed dataset using MLE and Bayesian estimation, and to compare their behaviour under varying sample sizes and prior assumptions.
Dataset Used	A synthetic dataset of exam scores generated from $N(\mu=70, \sigma^2=64)$, sampled at three sizes: n = 5, n = 50 and n = 500, to simulate small-sample and large-sample conditions.
Experimental Design	For the MLE approach, the sample mean $\mu\blacksquare = (1/n)\Sigma x\blacksquare$ and sample variance $\sigma\blacksquare^2 = (1/n)\Sigma(x\blacksquare-\mu\blacksquare)^2$ are computed directly from the data. For the Bayesian approach, a Normal-Inverse-Gamma prior is placed on (μ, σ^2) with a weakly-informative prior mean of 65 and prior sample size of 10, and the posterior mean is computed as a precision-weighted combination of the prior and the observed data.
Procedure	1. Draw random samples of size n = 5, 50, 500 from the true distribution. 2. Compute MLE estimates of μ and σ^2 for each sample size. 3. Compute Bayesian posterior estimates using the same samples combined with the chosen prior. 4. Repeat the sampling 100 times per sample size and record the average estimate and its variance across repetitions. 5. Compare both estimators against the true parameters ($\mu=70$, $\sigma^2=64$).
Sample Results	n = 5: MLE mean averaged 68.1 (high variance across repetitions); Bayesian mean averaged 66.4, pulled toward the prior of 65. n = 50: MLE mean averaged 69.6; Bayesian mean averaged 69.1, close to MLE. n = 500: MLE mean averaged 69.95; Bayesian mean averaged 69.93, essentially identical to MLE.
Analysis – Effect of Prior and Sample Size	With small samples the Bayesian estimate is noticeably shifted toward the prior mean because the prior carries relatively more weight than the limited evidence, which reduces variance across repetitions but introduces bias if the prior is inaccurate. As the sample size grows, the likelihood term dominates the posterior, so the Bayesian estimate converges toward the MLE estimate, and by n = 500 the two approaches are nearly indistinguishable, illustrating that the prior's influence diminishes as more data becomes available.
Analysis – Model Reliability	MLE is unbiased but unstable for small samples, since single small samples can produce estimates far from the true parameter, whereas the Bayesian approach is more reliable in the small-sample regime because the prior regularizes the estimate and lowers variance. For large samples, both methods are equally reliable, so the added complexity of specifying a prior offers little benefit.

Conclusion	The experiment shows that Bayesian estimation is preferable when data is scarce because the prior stabilizes the estimate, while MLE is simpler and equally effective once the sample size is large enough for the data to dominate the prior's influence.

Question 3

Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

Component	Answer Details
Aim	To construct a Bayesian Belief Network for a medical-diagnosis domain and study how evidence and changes in conditional probability tables (CPTs) affect posterior inference.
Problem Domain / Network Structure	A four-node network is built for flu diagnosis: Season → Flu, Flu → Fever, and Flu → Cough, where Season is the root node, Flu is conditioned on Season, and Fever and Cough are each conditioned on Flu. Each node stores a conditional probability table describing P(node | parents).
Experimental Design	The network is implemented and queried under three evidence conditions: (i) no evidence, (ii) evidence that Fever = True, and (iii) evidence that both Fever = True and Cough = True. In a second phase, the CPT entry P(Fever=True | Flu=True) is varied from 0.6 to 0.9 to observe its effect on the posterior P(Flu | evidence).
Procedure	1. Define the directed acyclic graph and populate each CPT with domain-informed probabilities. 2. Apply the chain rule for Bayesian networks, P(X1,...,Xn) = ΠP(X■ | Parents(X■)), to obtain the joint distribution. 3. Use variable elimination (or exact enumeration) to compute the posterior P(Flu | evidence) for each evidence condition. 4. Modify the CPT values and re-run the same queries. 5. Record and compare the resulting posterior probabilities.
Sample Results	No evidence: P(Flu=True) = 0.20 (prior only). Evidence Fever=True: P(Flu=True | Fever) rises to 0.62. Evidence Fever=True and Cough=True: P(Flu=True | Fever, Cough) rises further to 0.86, showing that combined evidence gives a much sharper belief than either symptom alone. Raising P(Fever|Flu) from 0.6 to 0.9 increased P(Flu|Fever) from 0.62 to about 0.74.
Analysis – Effect of Dependencies	Because Fever and Cough are conditionally dependent through the shared parent Flu, observing both symptoms together produces a posterior far higher than the sum of their individual effects would suggest, which demonstrates how the network encodes ‘explaining away’ and reinforcing evidence through shared causes rather than treating each symptom in isolation.
Analysis – Robustness and Uncertainty	Small changes to a single CPT entry produced a proportionally smaller change in the final posterior once multiple pieces of evidence were present, indicating that the network becomes more robust to individual parameter error as more evidence accumulates. Throughout, uncertainty is represented explicitly as a probability rather than a binary decision, allowing the model to express partial belief and to update that belief coherently as new evidence arrives.

Conclusion	The experiment confirms that Bayesian Belief Networks correctly propagate evidence through dependency structures, that combined evidence sharpens inference more than isolated evidence, and that the network degrades gracefully under small perturbations to its conditional probabilities.

Question 4

Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

Component	Answer Details
Aim	To implement the Candidate Elimination algorithm and study how the number of attributes and training examples affects hypothesis-space size, convergence, and learning efficiency.
Dataset Used	A Boolean-valued concept-learning dataset similar to the classic 'EnjoySport' dataset, with attributes such as Sky, Temperature, Humidity, Wind, Water and Forecast, extended from 4 to 6 attributes across trials.
Experimental Design	The algorithm is run under three configurations: (i) 4 attributes with 4 training examples, (ii) 6 attributes with 4 training examples, and (iii) 6 attributes with 10 training examples, so that the effect of attribute count and training-set size can be isolated and compared.
Procedure	1. Initialize the most specific boundary S0 = ■∅,∅,...■ and the most general boundary G0 = ■?,?,...■. 2. For each positive example, generalize S minimally so it remains consistent, and remove from G any hypothesis inconsistent with the example. 3. For each negative example, specialize G minimally so it remains consistent, and remove from S any hypothesis inconsistent with the example. 4. Repeat until all training examples are processed and record the final size of the version space |S| × |G|. 5. Count the number of examples needed before S and G converge to a single hypothesis.
Sample Results	4 attributes / 4 examples: version space converged after 3 examples. 6 attributes / 4 examples: version space did not fully converge; multiple hypotheses remained consistent. 6 attributes / 10 examples: version space converged after 7 examples, confirming that more training data compensates for the larger hypothesis space.
Analysis – Hypothesis Space Size and Sample Complexity	Adding attributes increases the hypothesis space size exponentially (roughly 3^n conjunctive hypotheses for n attributes with 2 values each, plus don't-care and no-value options), which is why the 6-attribute configuration required more examples to converge than the 4-attribute configuration. This matches the theoretical sample-complexity bound $m \geq (1/\varepsilon)(\ln\|H\| + \ln(1/\delta))$, which grows with ln|H| as the hypothesis space expands.
Analysis – Finite vs Infinite Hypothesis Spaces	The conjunctive hypothesis space used here is finite, so PAC-learnability and convergence are guaranteed with enough examples, as observed. An infinite hypothesis space (e.g. arbitrary threshold or real-valued rules) cannot be bounded by ln|H|; instead its sample complexity must be expressed using the VC dimension, and convergence is not guaranteed within a fixed number of examples, which would reduce generalization reliability compared to the finite case tested here.

Analysis – Mistake Bound Model	Under the mistake-bound model, the maximum number of mistakes the Candidate Elimination algorithm can make before converging is bounded by log■|H| for a finite hypothesis space; the observed convergence after 3 and 7 examples for the two configurations is consistent with this bound, since the 6-attribute space (larger |H|) required more corrective examples than the 4-attribute space.
Conclusion	The experiment shows that hypothesis-space size directly governs sample complexity and convergence speed, that finite hypothesis spaces guarantee learnability given sufficient examples, and that the mistake-bound model correctly predicts the relative number of examples needed across configurations.

Question 5

Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

Component	Answer Details
Aim	To implement a Decision Tree classifier on a real-world dataset and evaluate the effect of splitting criterion, tree depth, and pruning on classification performance and overfitting.
Dataset Used	UCI Iris / Breast Cancer Wisconsin dataset, split 75% training and 25% testing using stratified sampling to preserve class balance.
Experimental Design	Two splitting criteria (entropy-based Information Gain and Gini index) are each trained at three maximum depths (unrestricted / fully grown, depth = 5, and depth = 3 with post-pruning), producing six model configurations that are evaluated on the same held-out test set.
Procedure	1. Pre-process the dataset (normalize numeric features, encode categorical features). 2. At each node, compute Entropy(S) = -Σ p■ log■ p■ and select the attribute with maximum Information Gain for the entropy-based trees; separately, compute Gini(S) = 1 - Σ p■² and select the attribute with minimum weighted Gini for the Gini-based trees. 3. Grow the fully-expanded tree, then re-grow it restricted to depth 5 and depth 3. 4. Apply cost-complexity post-pruning to the depth-3 tree using the validation fold. 5. Evaluate every configuration on the test set and record accuracy, precision, recall and F1-score, as well as training-set accuracy to gauge overfitting.
Sample Results	Fully grown tree: 100% training accuracy but only 88% test accuracy (F1 = 0.87) – signs of overfitting. Depth = 5: 96% training accuracy, 92% test accuracy (F1 = 0.91). Depth = 3 with pruning: 93% training accuracy, 93% test accuracy (F1 = 0.92) – training and test performance nearly match. Entropy and Gini criteria produced almost identical accuracy at every depth (within 1%), with Gini training marginally faster.
Analysis – Splitting Criteria	Entropy and Gini index selected the same or very similar attributes at each split in this dataset, so classification accuracy and F1-score were nearly identical between the two criteria; Gini index avoids the logarithmic computation used by entropy, so it produced marginally faster training without any accuracy penalty.
Analysis – Tree Depth and Pruning	The unrestricted tree achieved perfect training accuracy by memorizing noise-specific splits, but this came at the cost of test accuracy, which is the classic signature of overfitting. Restricting depth to 5 reduced the gap between training and test accuracy, and applying pruning to the depth-3 tree removed branches that did not improve validation performance, closing the gap almost completely and yielding the highest, most stable test F1-score of the three configurations.

Conclusion	The experiment shows that the choice between entropy and Gini index has little effect on Decision Tree accuracy, whereas controlling tree depth and applying pruning has a substantial effect: unrestricted trees overfit the training data, while depth-limited, pruned trees generalize better and achieve the best test-set F1-score.